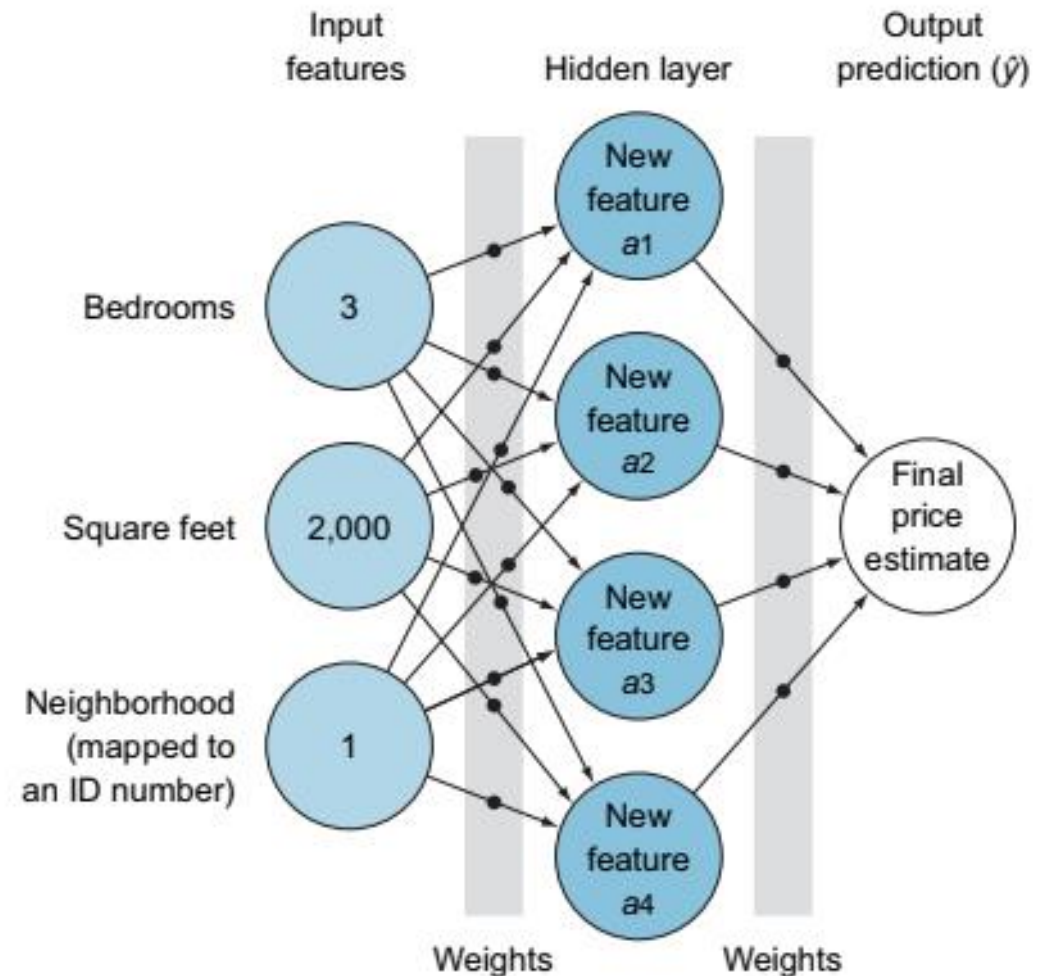


Some notes

The nodes in the hidden layers (a_i) are the new features that are learned after each layer. For example, if you look at figure 2.20, you see that we have three feature inputs (x_1 , x_2 , and x_3).

That is how a neural network learns new features: via the network's hidden layers.

First, they recognize patterns in the data. Then, they recognize patterns within patterns; then patterns within patterns within patterns, and so on. The deeper the network is, the more it learns about the training data.



Error functions (Error is always positive)

- Error functions can also be referred to as cost functions or loss functions, and these terms are used interchangeably in DL literature.
- The error function is a measure of how “wrong” the neural network prediction is with respect to the expected output (the label). It quantifies how far we are from the correct solution.
- For example, if we have a high loss, then our model is not doing a good job. The smaller the loss, the better the job the model is doing. The larger the loss, the more our model needs to be trained to increase its accuracy.
- Why need it ??? **Optimization problems** focus on defining an error function and trying to optimize its parameters to get the minimum error (more on optimization in the next section).
- In optimization problems, our ultimate goal is to find the optimum variables (weights) that would minimize the error function as much as we can. If we don't know how far from the target we are, how will we know what to change in the next iteration?
- The process of minimizing this error is called error function optimization.

Error functions

- Mean square error
- Mean squared error (MSE) is commonly used in regression problems that require the output to be a real value (like house pricing). Instead of just comparing the prediction output with the label ($\hat{y}_i - y_i$), the error is squared and averaged over the number of data points, as you see in this equation:

MSE is a good choice for a few reasons. The square ensures the error is always positive, and larger errors are penalized more than smaller errors. Also, it makes the math nice, which is always a plus.

$$E(W, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

MSE is quite sensitive to outliers, since it squares the error value. This might not be an issue for the specific problem that you are solving. In fact, this sensitivity to outliers might be beneficial in some cases. For example, if you are predicting a stock price, you would want to take outliers into account, and sensitivity to outliers would be a good thing.

Notation	Meaning
$E(W, b)$	The loss function. Is also annotated as $J(W, b)$ in other literature.
W	Weights matrix. In some literature, the weights are denoted by the theta sign (θ).
b	Biases vector.
N	Number of training examples.
$\hat{y}_i$	Prediction output. Also notated as $h_{w, b}(X)$ in some DL literature.
y_i	The correct output (the label).
$(\hat{y}_i - y_i)$	Usually called the <i>residual</i> .

In that case, you are more interested in the median and less in the mean. A variation error function of MSE called mean absolute error (MAE) was developed just for this purpose. It averages the absolute error over the entire dataset without taking the square of the error:

$$E(W, b) = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

Error functions

Cross-entropy

- Cross-entropy is commonly used in classification problems because it quantifies the difference between two probability distributions.
- For example, suppose that for a specific training instance, we are trying to classify a dog image out of three possible classes (dogs, cats, fish). The true distribution for this training instance is as follows:

Probability(cat)	P(dog)	P(fish)
0.0	1.0	0.0

- We can interpret this “true” distribution to mean that the training instance has 0% probability of being class A, 100% probability of being class B, and 0% probability of being class C. Now, suppose our machine learning algorithm predicts the following probability distribution:

Probability(cat)	P(dog)	P(fish)
0.2	0.3	0.5

How close is the predicted distribution to the true distribution? That is what the cross entropy loss function determines. We can use this formula:

$$E(W, b) = -\sum_{i=1}^m \hat{y}_i \log(p_i)$$

where (y) is the target probability, (p) is the predicted probability, and (m) is the number of classes. The sum is over the three classes: cat, dog, and fish. In this case, the loss is 1.2:

$$E = - (0.0 * \log(0.2) + 1.0 * \log(0.3) + 0.0 * \log(0.5)) = 1.2$$

So that is how “wrong” or “far away” our prediction is from the true distribution.

Error functions

- In later iterations, the network learns some patterns and gets the predictions a little better, up to 50%:

Probability(cat)	P(dog)	P(fish)
0.3	0.5	0.2

- Then we calculate the loss again:
- $E = -(0.0 \cdot \log(0.3) + 1.0 \cdot \log(0.5) + 0.0 \cdot \log(0.2)) = 0.69$
- You see that when the network makes a better prediction (dog is up to 50% from 30%), the loss decreases from 1.2 to 0.69. In the ideal case, when the network predicts that the image is 100% likely to be a dog, the cross-entropy loss will be 0.

Suppose the input $x = 0.3$, and its label (goal prediction) $y = 0.8$. The prediction out- put ($\hat{y}$) of this perception is calculated as follows:

$$\hat{y}_i = w \cdot x = w \cdot 0.3$$

And the error, in its simplest form, is calculated by comparing the prediction $\hat{y}$ and the label y :

$$\text{error} = |\hat{y} - y|$$

$$= |(w \cdot x) - y|$$

$$= |w \cdot 0.3 - 0.8|$$

- If you look at this error function, you will notice that the input (x) and the goal prediction (y) are fixed values. They will never change for these specific data points. The only two variables that we can change in this equation are the error and the weight. Now, if we want to get to the minimum error, which variable can we play with? Correct: the weight! The weight acts as a knob that the network needs to adjust up and down until it gets the minimum error.

The process of finding the goal weights of the neural network happens by adjusting the weight values in an iterative process using an *optimization algorithm*.

Optimization algorithms

- Finally, the neural network needs to adjust the weights (on all edges) until it gets the minimum error value, which means maximum accuracy. Now, all we need to do is build algorithms that can find the optimum weights for us.

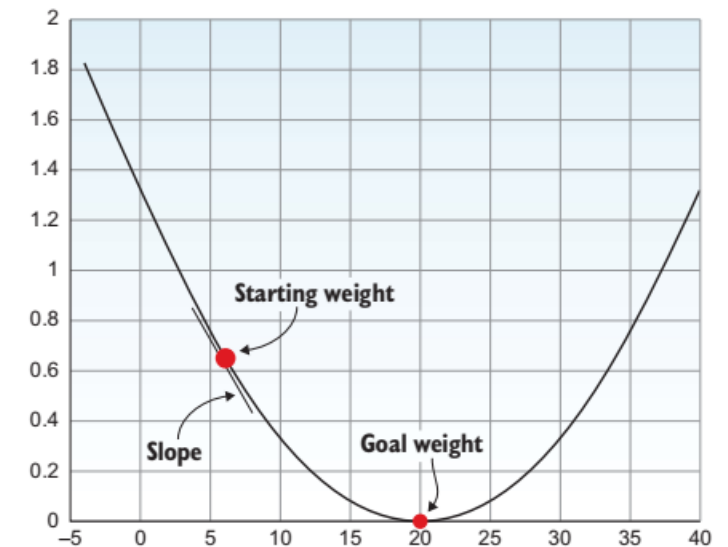
Optimization is a way of framing a problem to maximize or minimize some value. The best thing about computing an error function is that we turn the neural network into an optimization problem where our goal is to minimize the error.

In neural networks, optimizing the error function means updating the weights and biases until we find the optimal weights, or the best values for the weights to produce the minimum error.

In a neural network of the simplest form, a perceptron with one input, we have only one weight. We can easily plot the error (that we are trying to minimize) with respect to this weight, represented by the 2D curve in figure.

But what if we have two weights? If we graph all the possible values of the two weights, we get a 3D plane of the error .

What about more than two weights? Your network will probably have hundreds or thousands of weights (because each edge in your network has its own weight value).



Optimization algorithms

- Why do we need an optimization algorithm? Can't we just brute-force through a lot of weight values until we get the minimum error?
- Suppose we used a brute-force approach where we just tried a lot of different possible weights (say 1,000 values) and found the weight that produced the minimum error. Could that work? Well, theoretically, **yes**.
- If we have 20 edges (weights) from the input to the hidden layer, plus 5 weights from the hidden layer to the output prediction, totaling 25 weight variables that need to be adjusted for optimum values. To brute-force our way through a simple neural network of this size, if we are trying 1,000 different values for each weight, then we will have a total of 1075 combinations:

$$1,000 \times 1,000 \times \dots \times 1,000 = 1,000^{25} = 10^{75} \text{ combinations}$$

supercomputer would need

$$\frac{10^{75}}{93 \times 10^{15}} = 1.08 \times 10^{58} \text{ seconds} = 3.42 \times 10^{50} \text{ years}$$

So we have to come up with a different approach to find the optimal weights

gradient descent algorithm

Batch gradient descent

- The general definition of a gradient (also known as a derivative) is that it is the function that tells you the slope or rate of change of the line that is tangent to the curve at any given point. It is just a fancy term for the slope or steepness of the curve

Gradient descent simply means updating the weights iteratively **to descend the slope of the error curve until we get to the point with minimum error.**

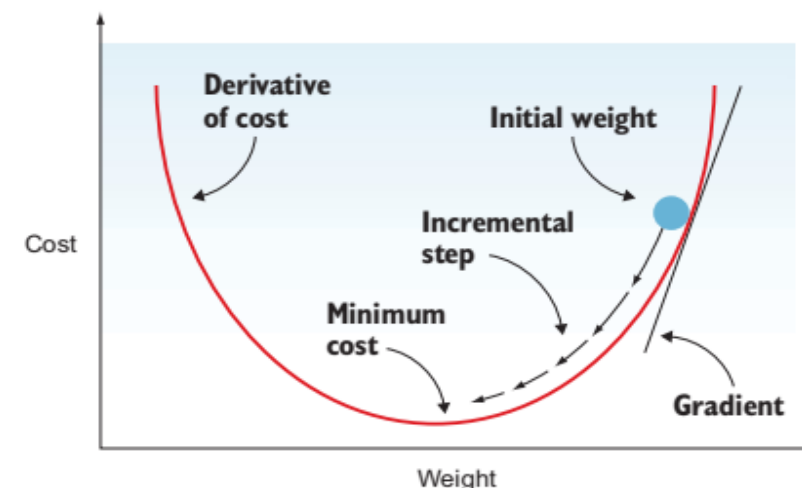
Let's take a look at the error function that we introduced earlier with respect to the weights. At the initial weight point, we calculate the derivative of the error function to get the slope (direction) of the next step.

We keep repeating this process to take steps down the curve until we reach the minimum error

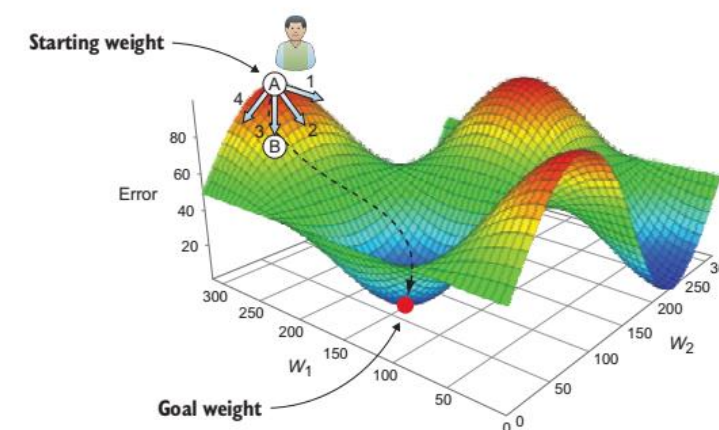
The random initial weight (starting weight) is at point A, and our goal is to descend this error mountain to the goal w_1 and w_2 weight values, which produce the minimum error value. The way we do that is by taking a series of steps down the curve until we get the minimum error.

In order to descend the error mountain, we need to determine two things for each step:

- The step direction (gradient)
- The step size (learning rate)



Gradient descent takes incremental steps to descend the error function



Batch gradient descent

THE DIRECTION (GRADIENT)

This process is called gradient descent. By taking the derivative of the error with respect to the weight ($\frac{dE}{dw}$), we get the direction that we should take. Now there's one thing left. The gradient only determines the direction. How large should the size of the step be? It could be a 1-foot step or a 100-foot jump. This is what we need to determine next.

THE STEP SIZE (LEARNING RATE α)

The learning rate is the size of each step the network takes when it descends the error mountain, and it is usually denoted by the Greek letter alpha (α). It is one of the **most important hyper-parameters that you tune** when you train your neural network (more on that later).

A larger learning rate means the network will learn faster (since it is descending the mountain with larger steps), and smaller steps mean slower learning.

$$\Delta w_i = -\alpha \frac{dE}{dw_i}$$

PUTTING DIRECTION AND STEP TOGETHER

$$w_{\text{next-step}} = w_{\text{current}} + \Delta w$$

Backpropagation

Backpropagation is the core of how neural networks learn. Up until this point, you learned that training a neural network typically happens by the repetition of the following three steps:

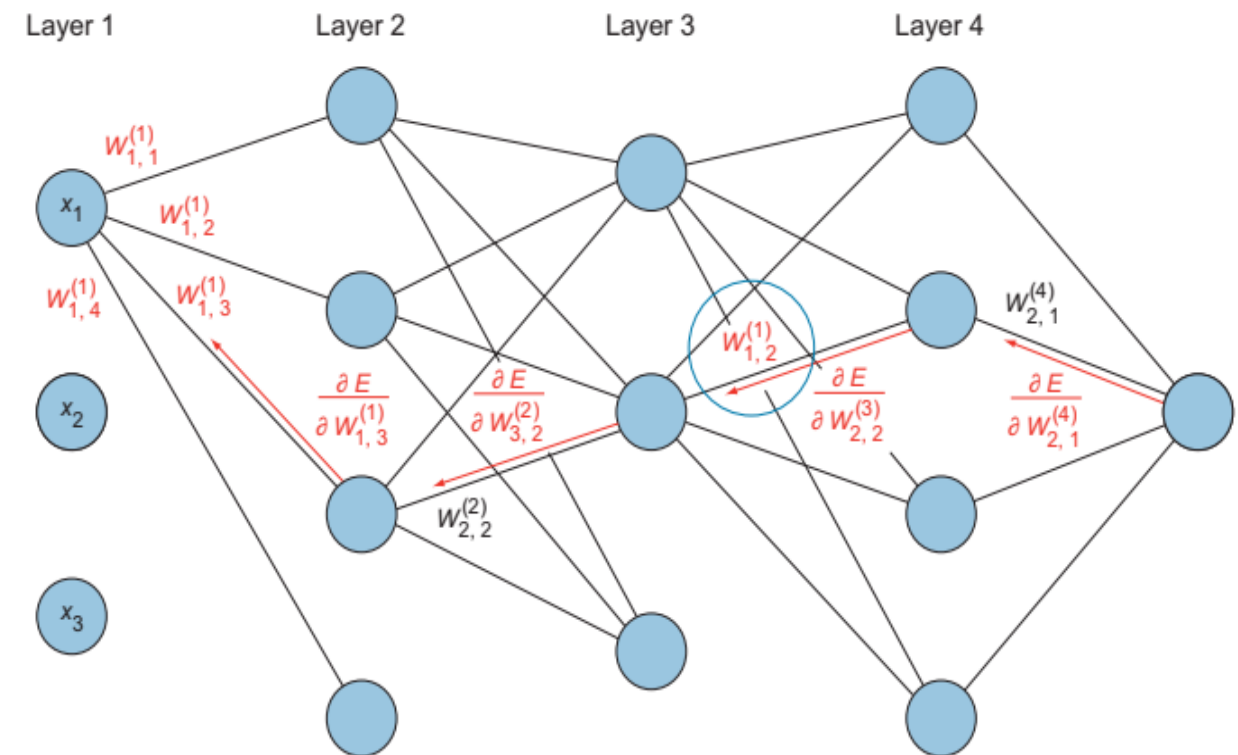
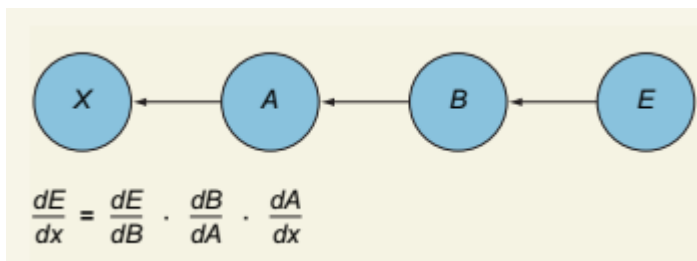
- Feedforward
- calculate the error or loss function
- Use a gradient descent optimization algorithm to compute the Δw that optimizes the error function:

Back propagate the Δw through the network to update the weights:

$$W_{\text{new}} = W_{\text{old}} - \alpha \left(\frac{\partial \text{Error}}{\partial W_x} \right)$$

Labels in the diagram:
 - W_{new} : New weight
 - W_{old} : Old weight
 - α : Learning rate
 - $\frac{\partial \text{Error}}{\partial W_x}$: Derivative of error with respect to weight

Backpropagation, or backward pass, means propagating derivatives of the error with respect to each specific weight to compute the derivatives of the total error with respect to the weights all the way back to the input, we need a calculus rule called the chain rule.



$$\frac{dE}{dw_{1,3}^{(1)}} = \frac{dE}{dw_{2,1}^{(4)}} \times \frac{dw_{2,1}^{(4)}}{dw_{2,2}^{(3)}} \times \frac{dw_{2,2}^{(3)}}{dw_{3,2}^{(2)}} \times \frac{dw_{3,2}^{(2)}}{dw_{1,3}^{(1)}}$$

Batch gradient descent

```
opt = keras.optimizers.SGD(learning_rate=0.01)
```

```
model.compile(loss='categorical_crossentropy', optimizer=opt)
```

```
model.fit(train_ds, epochs=50, validation_data=val_ds)
```

Disadvantages of Batch Gradient Descent?
What the solution??